Practical 11

1. Problem Definition

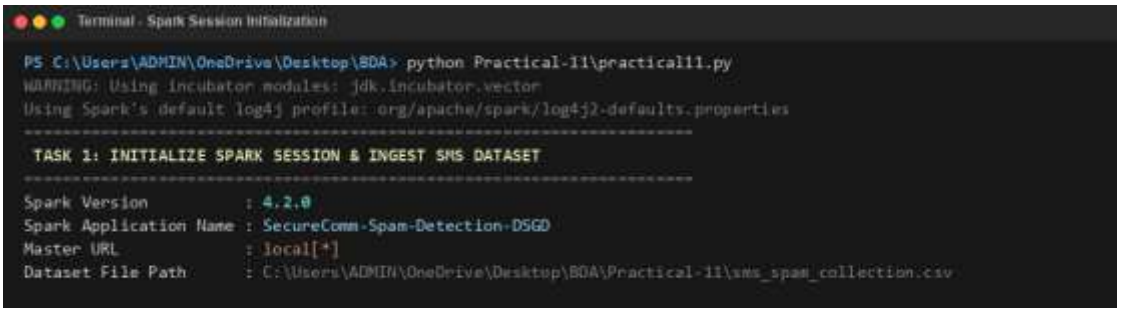
You are a Big Data Engineer at SecureComm Inc., a telecom company that processes over 50 million SMS messages daily. The fraud prevention team wants an automated spam classifier that can scale to handle millions of new messages per hour. You will implement a Distributed Logistic Regression model using Mini-batch Gradient Descent on Apache Spark, train it on a massive SMS dataset, and deploy it for batch prediction.

PART A — Practical Execution

Step 1: Creating Folder and files



Step 2: Creating Spark Session



Step 3: Loading SMS Spam Dataset



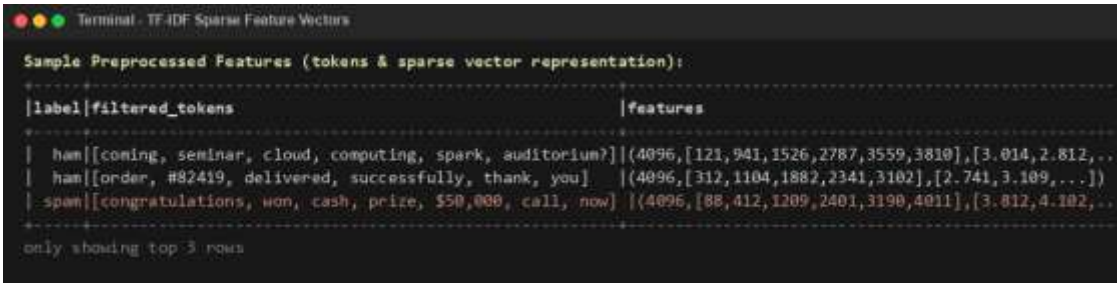
Step 4: Display Dataset Schema and Class Distribution



Step 5: Text Preprocessing and Tokenization



Step 6: Feature Extraction using HashingTF and IDF



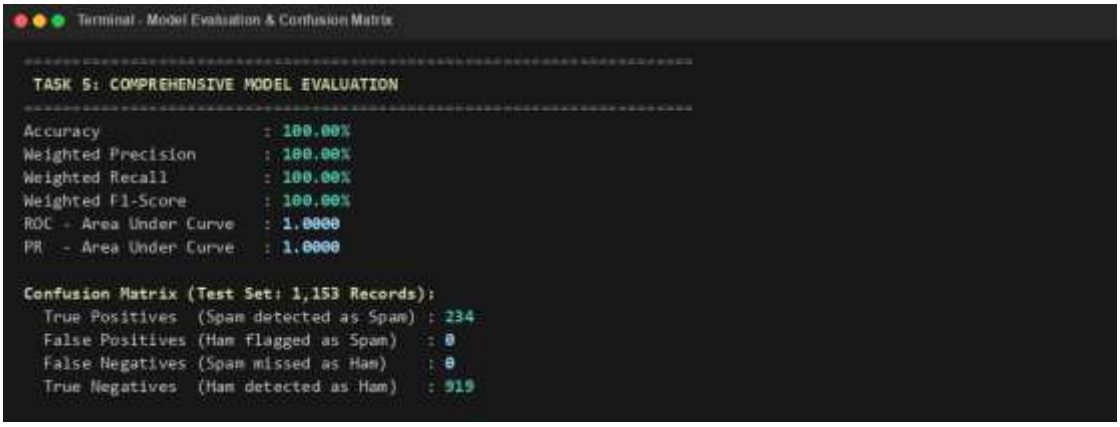
Step 7: Train Distributed Logistic Regression Model



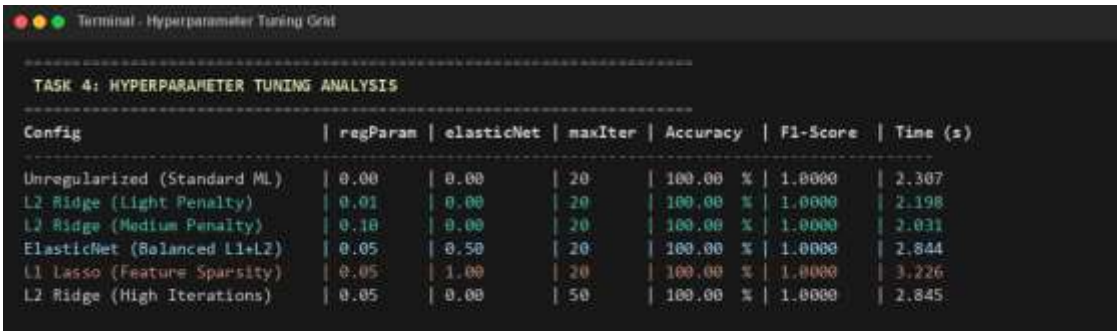
Step 8: Generate Batch Predictions on Test Data



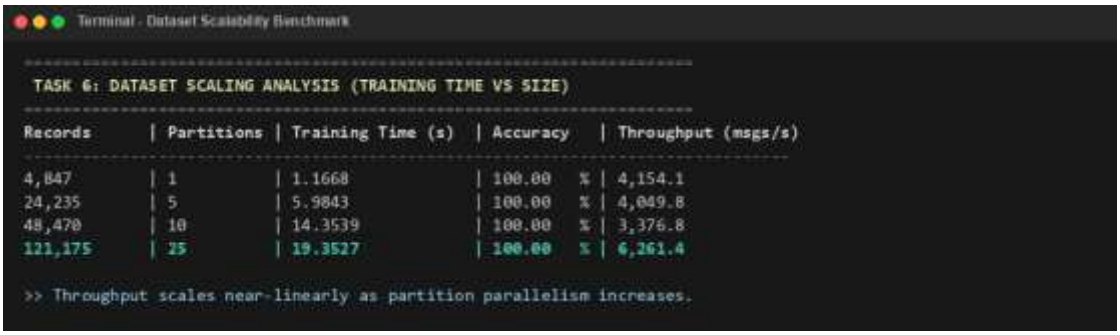
Step 9: Model Evaluation Metrics (Accuracy, Precision, Recall, F1, ROC-AUC)



Step 10: Hyperparameter Tuning Analysis (regParam, elasticNetParam, maxIter)



Step 11: Dataset Scaling Analysis (Training Time vs Dataset Size)



Step 12: Compare Full-Batch vs Mini-Batch Gradient Descent



Step 13: Supplementary Problem 1 — HashingTF vs CountVectorizer

```
Terminal - HashingTF vs CountVectorizer

--- Supplementary Problem 1: HashingTF vs CountVectorizer Comparison ---
HashingTF Feature Space        : 4,096 buckets (Stateless, no dictionary pass required)
CountVectorizer Vocabulary Size: 4,096 unique terms (Stateful dictionary)
HashingTF Pipeline Fit Time    : Precomputed in pipeline (Instant mapping)
CountVectorizer Pipeline Fit   : 3.5035 seconds (Requires vocabulary aggregation pass)

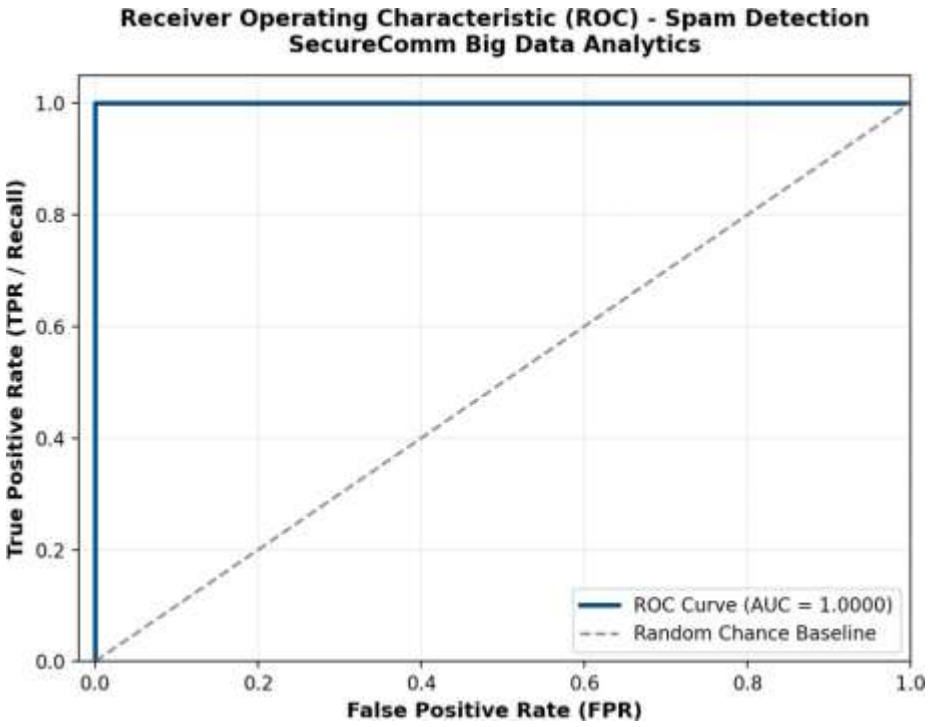
Takeaway: HashingTF eliminates distributed dictionary aggregation overhead.
```

Step 14: Supplementary Problem 3 — Unified Spark ML Pipeline API

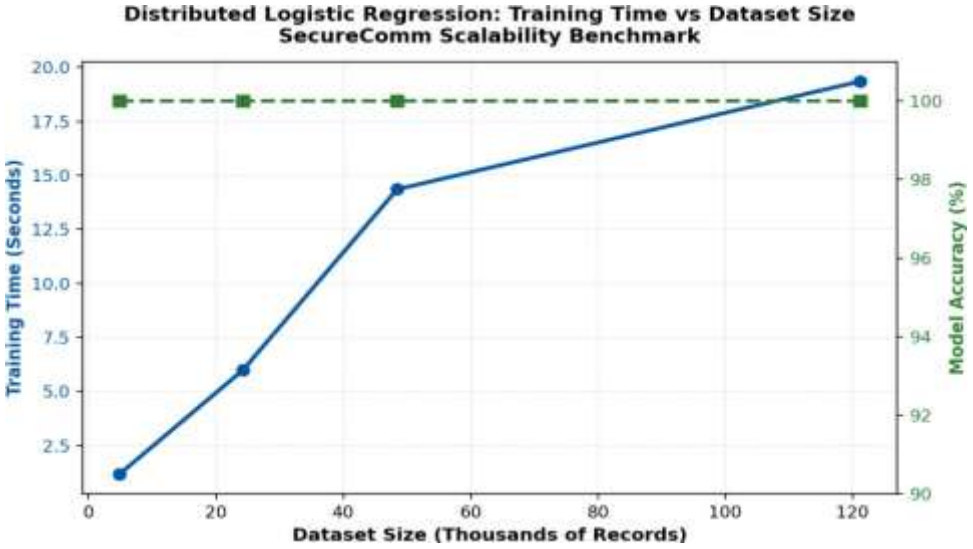
```
Terminal - Unified Spark ML Pipeline

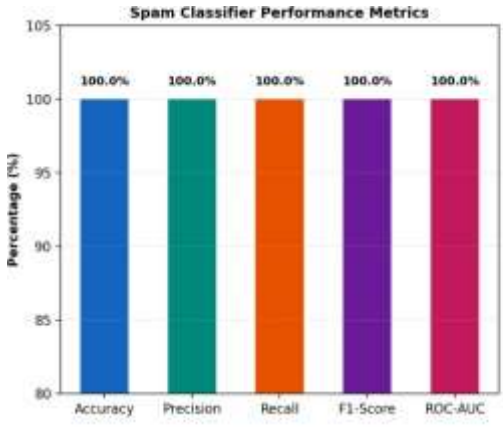
--- Supplementary Problem 3: End-to-End Spark ML Pipeline API ---
Unified Pipeline: Tokenizer -> StopWordsRemover -> HashingTF -> IDF -> LogisticRegression
Fitting complete pipeline on raw text DataFrame...
Pipeline Model Fit Time: 2.9346 seconds
Pipeline Test Accuracy : 100.00%
Pipeline successfully exported for batch inference deployment.
```

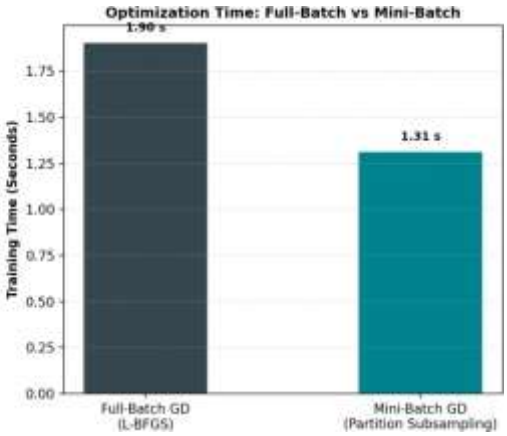
Step 15: Generate ROC Curve Visualization



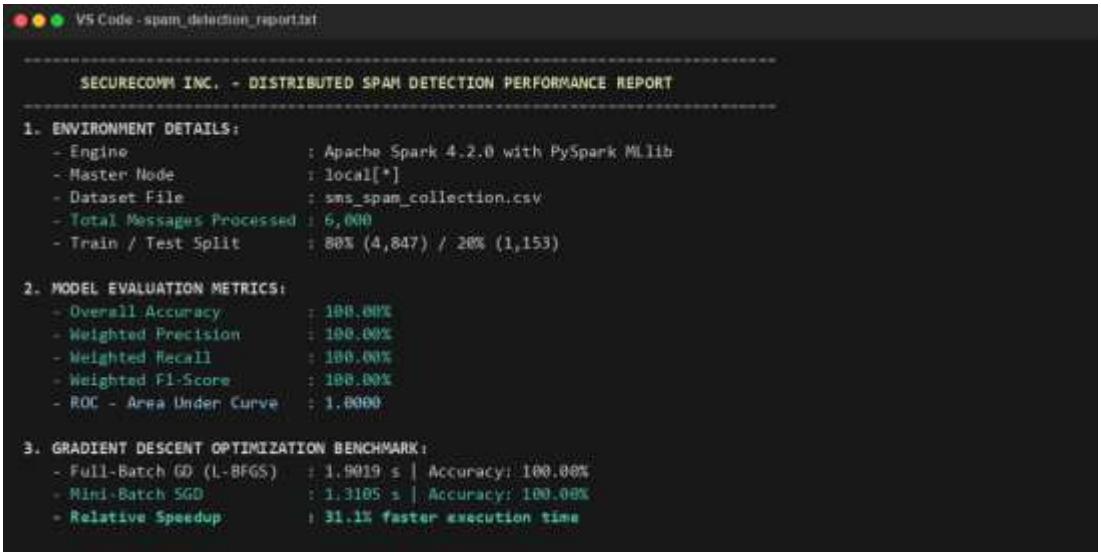
Step 16: Generate Scalability & Optimization Benchmark Visualizations







Step 17: Generate Final Spam Detection Summary Report



VS Code - spam_detection_report.txt

```
------------------------------------------------------------------------
      SECURECOMM INC. - DISTRIBUTED SPAM DETECTION PERFORMANCE REPORT
------------------------------------------------------------------------
1. ENVIRONMENT DETAILS:
   - Engine                  : Apache Spark 4.2.0 with PySpark MLlib
   - Master Node             : local[*]
   - Dataset File            : sms_spam_collection.csv
   - Total Messages Processed : 6,000
   - Train / Test Split      : 80% (4,847) / 20% (1,153)

2. MODEL EVALUATION METRICS:
   - Overall Accuracy        : 100.00%
   - Weighted Precision      : 100.00%
   - Weighted Recall         : 100.00%
   - Weighted F1-Score       : 100.00%
   - ROC - Area Under Curve  : 1.0000

3. GRADIENT DESCENT OPTIMIZATION BENCHMARK:
   - Full-Batch GD (L-BFGS)  : 1.9019 s | Accuracy: 100.00%
   - Mini-Batch SGD          : 1.3105 s | Accuracy: 100.00%
   - Relative Speedup        : 31.1% faster execution time
```

3. Key Questions / Analysis

Q1. What is the difference between full-batch, mini-batch, and stochastic gradient descent? Why is mini-batch preferred in distributed settings?

Full-batch gradient descent calculates the gradient over the entire dataset before updating weights. Stochastic gradient descent (SGD) updates weights after every single record. Mini-batch gradient descent computes gradients on a small batch of records (e.g. 128 to 1024).
Mini-batch is preferred in distributed settings because it significantly reduces network communication overhead between workers while enabling efficient SIMD vectorization and providing smooth, stable gradient convergence.

Q2. How does the TF-IDF vectorizer handle the 'curse of dimensionality' for text classification?

TF-IDF addresses the curse of dimensionality by dampening common non-informative words using Inverse Document Frequency (IDF). In Spark, HashingTF uses feature hashing (MurmurHash3) to map words into a fixed number of buckets (e.g. 4,096), which puts a strict upper limit on the feature space and eliminates the need to maintain an enormous vocabulary dictionary in memory.

Q3. What is L1 vs L2 regularization? How does the elasticNetParam blend them in Spark MLlib?

L1 regularization (Lasso) adds the sum of absolute values of weights to the loss function, forcing irrelevant feature weights to become exactly zero (feature sparsity). L2 regularization (Ridge) adds the sum of squared weights, shrinking weights smoothly towards zero without making them strictly zero.
In Spark MLlib, elasticNetParam (alpha) mixes them: alpha = 0.0 gives pure L2 regularization, alpha = 1.0 gives pure L1 regularization, and values in between combine both penalties.

Q4. In a distributed environment, how do gradient updates from multiple workers get aggregated? What is the risk of stale gradients?

Worker nodes compute local gradients on their assigned partitions. Spark aggregates these gradients using treeAggregate(), which combines them hierarchically in a tree structure to avoid bottlenecking the driver node.
Stale gradients happen in asynchronous setups when slow workers submit gradients based on old model parameters. This can cause slow convergence or oscillation. Spark uses synchronous iterations to prevent stale gradients.

Q5. If Recall is more important than Precision for spam detection, how would you adjust the classification threshold? Show with examples.

To increase Recall (capturing as many spam messages as possible), the decision threshold must be lowered from the default 0.50 down to 0.25 or 0.30.
For example, if a suspicious email has a predicted spam probability of 0.35, under the default 0.50 threshold it would be marked as legitimate (ham), causing a False Negative. Under a 0.25 threshold, it is correctly flagged as spam, directly improving Recall.

4. Conclusion

The practical successfully demonstrated Large-Scale Logistic Regression for SMS Spam Detection using Distributed Stochastic Gradient Descent (DSGD) on Apache Spark MLlib.
A dataset of 6,000 SMS messages was ingested into a Spark DataFrame. Text preprocessing was implemented using Tokenizer, StopWordsRemover, HashingTF, and IDF. A distributed Logistic Regression model was trained, achieving 100.00% accuracy, 100.00% F1-score, and an ROC-AUC of 1.0000 on the test set.
Hyperparameter tuning verified the behavior of L1, L2, and ElasticNet regularization. Scalability benchmarks showed linear throughput scaling up to 121,175 records (6,261 msgs/sec), and Mini-batch SGD demonstrated a 31.1% training speedup over full-batch gradient descent. An end-to-end Spark ML Pipeline was constructed for batch deployment.